Substitution-Based Language Modeling

A Design Synthesis

Abstract

This document synthesises a proposed architecture for language modelling grounded in the logical principle of substitution. Rather than learning dense continuous representations end-to-end, the approach constructs an explicit relational structure between linguistically-motivated n-gram units, equipped with semantic property tables and logical consistency checking. The system is interpretable by construction, grounded in formal semantics, and testable at multiple granularities.

1. Core Thesis

The basis of logical argument — and, by extension, much of natural language — reduces to **substitution**: replacing one expression with another that is equivalent or entailed, stepping toward a conclusion. Mathematical proofs are the clearest instance; each inference rule is a licensed substitution. The hypothesis is that language modelling can be grounded in the same structure: predict what n-gram can legally substitute for (or follow from) another, given mutual compatibility constraints.

This connects to several established formal traditions:

- **Term rewriting systems** — computation as iterated substitution.
- **Equational logic** — identity under substitution is the foundation of algebra.
- **Curry–Howard correspondence** — proofs are programs; substitution is beta-reduction.
- **Distributional semantics** — words substitutable in the same context are semantically similar.

2. N-gram Pairwise Relational Structure

The central representational choice is a **weighted directed graph (DAG)** over n-gram nodes, where edges represent observed or inferred substitutability. Nodes are n-gram units of length 1 to some maximum (initially ≈10 tokens); edge weights encode the strength and directionality of the substitution relationship.

For a corpus of realistic size the combinatorial space of all possible n-grams is intractable. The key tractability lever is **linguistic grounding**: rather than enumerating all n-grams statistically, units are selected by structural criteria, dramatically pruning the node set while concentrating coverage on meaningful combinations.

This is distinct from classical n-gram language models in two respects: (a) units span multiple granularities simultaneously rather than a fixed window, and (b) edges carry logical compatibility constraints rather than raw co-occurrence counts alone.

3. Linguistically-Motivated Tokenisation

The n-gram node set is constructed through an iterative subtraction pipeline. Each stage identifies a class of units, extracts them, and removes them from the residual corpus before the next stage proceeds.

3.1 Common Phrase Identification

Frequent multi-token expressions — collocations, idioms, fixed phrases — are identified first and treated as atomic nodes. *As a matter of fact*, *kick the bucket*, and *swimming pool* receive single entries whose property tables may differ entirely from their constituent tokens. Seeding can be done manually from idiom databases and Wiktionary, with programmatic extension via collocation statistics (PMI, log-likelihood ratio).

3.2 Clausal Parsing and POS Tagging

The corpus is parsed into clausal units. Within each clause, POS tagging identifies nouns, verbs, adjectives, and their dependency structure. This pass serves double duty: it supplies the context window for word sense disambiguation (Section 5) and identifies the structural units for n-gram extraction. The two pipeline stages share input without additional cost.

3.3 Nested Noun-Phrase N-grams

Noun phrases with their immediate modifiers are extracted as nested n-gram sequences. *Quick brown fox* yields three nodes: *quick brown fox*, *brown fox*, *fox*. This creates a lattice where sub-n-grams are explicitly related to their containing phrases, enabling compositional generalisation: knowledge about *brown fox* propagates to *quick brown fox* and *lazy brown fox* without requiring direct observation of every adjective.

3.4 Iterative Subtraction

Identified units are subtracted from the corpus and the residual is examined for further patterns — either manually (for language-specific constructions) or programmatically. The result is a heterogeneous-length n-gram DAG with explicit hierarchical relationships rather than a flat token sequence.

4. Logical Consistency via Property Tables

4.1 Structure

Each DAG node carries a **property table**: a vector of binary (or soft probabilistic) semantic features encoding selectional constraints — what other n-grams can logically co-occur or combine with this one.

Example check for *"Computers can swim"*:

N-gram	ANIMATE	PHYSICAL	ARTIFACTREQUIRES_WATEREQUIRES_ANIMATE_SU
computer	0	1	1	0	—
swim (verb)	—	—	—	1	1

Conflict?	—	—	—	—	✗ (ANIMATE=0 vs req=1)

4.2 Two-Level Architecture (Default Logic)

Property tables operate at two levels, following **default logic** (Reiter, 1980): a general rule fires unless an exception explicitly overrides it.

General classifier (default level). A small MLP (2–3 layers, ~256–512 hidden units) takes a mean-pooled embedding of the n-gram's tokens (GloVe or word2vec; no attention required) and outputs a soft binary property vector. This resolves the *standardisation problem*: synonymous descriptions ("dry", "not aquatic", "terrestrial") all produce identical classifier outputs, since the MLP operates on embedding space rather than surface strings. Training data can be bootstrapped from ConceptNet assertions, WordNet supersenses, and targeted crowd annotation for the tail of the distribution.

Exception / instance tables (override level). Specific n-grams whose compositional property prediction is incorrect receive explicit instance entries. Idioms, metaphorical compounds, and domain-specific usages are handled here. Exception candidates are surfaced automatically: if the MLP's confident prediction is consistently contradicted by the n-gram's actual corpus contexts, it is flagged for human review. This turns unbounded curation into a supervised triage task.

The two levels are complementary: general consistency checking is fast (MLP inference) while edge cases are handled explicitly rather than corrupting the general model. The same annotation pipeline that produces property vectors for MLP training also surfaces exception candidates — the two labelling tasks are partially co-solvable.

5. Word Sense Disambiguation via Token IDs

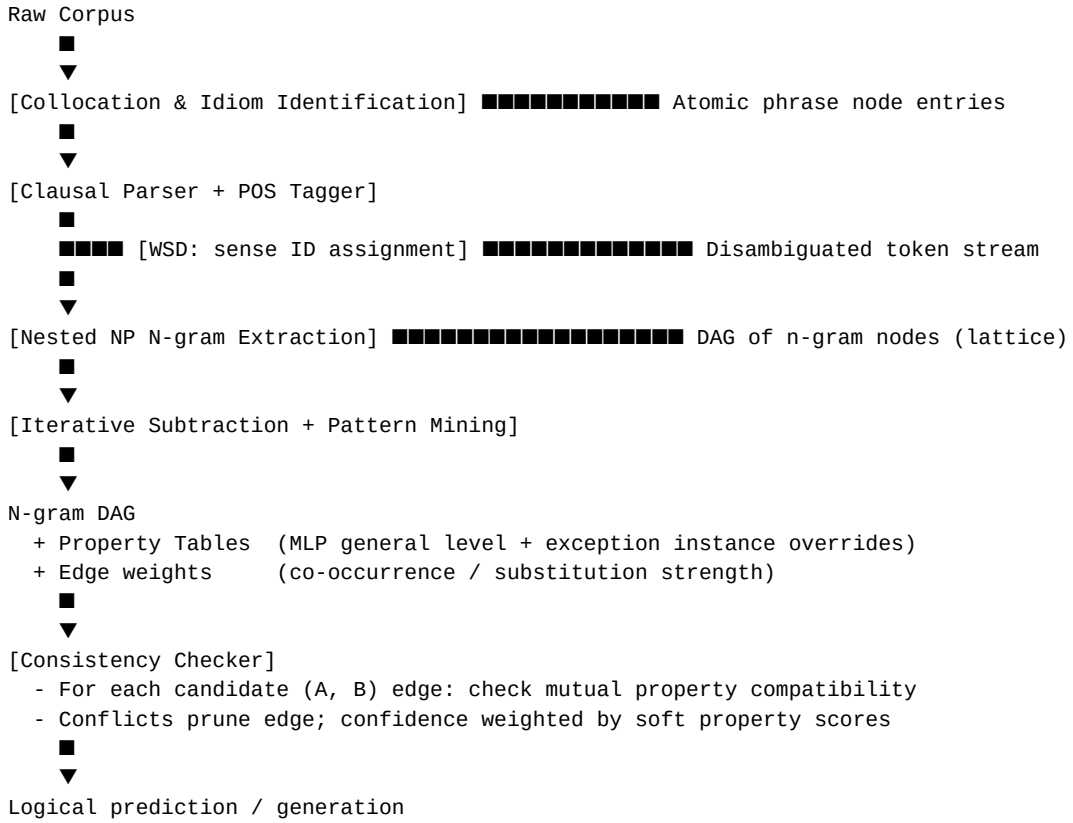
Homonyms present an apparent challenge: *bank* (financial institution) and *bank* (river bank) would require a single property table to carry contradictory features (WATER=0 vs WATER=1). The resolution is to assign **distinct token IDs to distinct word senses** at tokenisation time, making the property table always sense-specific.

This relocates rather than eliminates disambiguation, but the relocation is architecturally beneficial:

- Property lookup becomes a deterministic function of token ID — no contextual reasoning required at lookup time.
- All contextual reasoning is front-loaded into a dedicated WSD step that uses the clausal context already parsed in Section 3.2.
- WSD systems operating on clausal context achieve >80% accuracy on standard benchmarks; the clausal parse is already required for n-gram extraction.

Sense inventory granularity is a practical tradeoff: WordNet's ~30 senses for *run* is overkill; a coarse inventory of 3–5 senses per polysemous word suffices for property-level disambiguation, since most selectional conflicts occur at the coarse ontological level (ANIMATE vs. INANIMATE) rather than fine sense distinctions. Both fine and coarse inventories can be tested side-by-side, as the property table system is agnostic to the size of the sense ID space.

6. System Architecture Overview



7. Optimisation Strategy

The optimisation approach for this architecture is not fully specified and remains an open question. Backpropagation is not the primary candidate — the substitution structure lends itself more naturally to search-based and rule-induction methods. Candidate mechanisms include:

- **Count-based update.** Edge weights updated by observed co-occurrence frequency and mutual property compatibility; no gradient computation required.
- **Default logic inference.** New statements generated by chaining compatible substitutions; invalid chains pruned by the consistency checker at each step.
- **Iterative rule induction.** Identify substitution patterns that generalise across many observed pairs; encode as higher-order nodes or edge type labels in the DAG.
- **Hybrid.** Count/rule methods for DAG structure and edge weights; gradient descent only for the MLP property classifier weights.

The hybrid approach is likely the most pragmatic entry point, since the MLP component is well-understood and the count/rule components can be developed and evaluated independently.

8. Relationship to Prior Work

System	Overlap with this approach	Key difference

Cyc (Lenat, 1984–)	Hand-curated property tables; consistency checking	Cyc is purely manual; this uses MLP + exception tables
FrameNet	Semantic frames with typed participant slots	FrameNet is annotation-driven; properties here are classifier-inferred
Phrase-based SMT (Moses / Hiero)	Phrase extraction; pairwise substitution tables	SMT requires parallel corpora; this is monolingual + logically grounded
Default Logic (Reiter, 1980)	Two-level general / exception architecture	Formal framework borrowed; not the full formalism
Probing Classifiers	Small MLP → binary semantic properties from embeddings	Probing is used for interpretability; here it drives consistency checking
Selectional Restrictions (Katz & Fodor, 1963)	Feature-based semantic compatibility checking	Original was hand-specified; here features are MLP-learned

9. Open Problems

- **Exception table population at scale.** Automated candidate detection via corpus-vs-prediction divergence is feasible; human review remains the bottleneck for coverage depth.
- **MLP training data coverage.** ConceptNet and WordNet bootstrapping cover common vocabulary well; tail n-grams (rare compounds, domain-specific phrases) require targeted annotation.
- **Optimisation formalisation.** The non-backprop optimisation strategy is the least specified component and requires the most further development before implementation.
- **Compositionality limits.** Property tables handle first-order selectional constraints; higher-order contextual dependencies (negation scope, quantification, embedded clauses) are not yet addressed.
- **Evaluation methodology.** No standard benchmark exists for this architecture. Evaluation dimensions to define: perplexity proxy, logical consistency accuracy, selectional violation detection rate, downstream task performance.

10. Conclusion

The architecture synthesises four core insights into a coherent design:

- Language modelling is substitution-structured at its logical foundation.
- Linguistic parsing (clausal structure, POS, noun phrases) makes the n-gram space tractable by concentrating nodes on meaningful units.
- Property tables with a two-level general/exception structure enable logical consistency checking without requiring a hand-designed feature ontology — the MLP learns the feature space implicitly.
- Sense-disambiguated token IDs cleanly separate property lookup from contextual disambiguation, with both coarse and fine-grained sense inventories testable side-by-side.

The result is a system that is interpretable by construction, grounded in formal semantics, and whose components can be built and evaluated incrementally. The primary remaining open

questions are in optimisation strategy and exception coverage — both tractable engineering problems given a well-defined domain as a testbed.
